NETAJI SUBHAS INSTITUTE OF TECHNOLOGY, BIHTA

Approved by AICTE: Affiliated to B.E.U. & S.B.T.E, Bihar

Board Roll No..

Name : Suraj Arya

Session : 2024-28

Year/Semester: 2nd / 3rd Roll No. 241064

Branch : Computer Science Engineering

Subject : Operating System

EXTERNAL LAB RECORD

Date:

Experiment no. 1

Page No.:

⇒ **Aim:-**

Write the list of basic linux commands

⇒ **Requirements:-**

* Linux OS (Ubuntu / Fedora / Debian etc.)
* Terminal access.

⇒ Basic Linux Commands list

* **Directory commands.**

1. pwd:- print current working directory
2. ls:- List files and directories.
3. cd:- change directory
4. mkdir:- Create new directory.
5. rmdir:- Remove empty directory

* **File Commands:-**

1. touch:- Create empty file.
2. cat:- display file content.
3. cp:- Copy file.
4. mv:- move or rename file.
5. rm:- Delete file.

* **viewing & Editing Commands.**

1. more:- view file page by page
2. less:- view file with scroll support
3. nano:- open nano editor
4. vi/vim:- open vi editor.

```
ct [0] = 0;

for (int i= 0; i<n; i++)
{ ct [i] + = bt [i];
  ct [i+1] = tat [i] = ct [i];
  wt [i] = tat [i] - bt [i];
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0; i<n; i++)
{ printf("%d\t%d\t%d\t%d\t%d\t%d\n",
          pid [i], at[i], bt[i], ct[i], tat[i],wt[i]);
}

int sum = 0, sum2= 0;
for (int i =0; i<n; i++)
{ sum += tat [i];
  sum2 += wt [i];
}

printf ("Average Turn Around Time is %.2f.\n",
         (float) sum /n);

printf ("Average Waiting Time is %.2f. ",
         (float) sum2 /n);
return 0;

}
```

AIM:- To implement SJF CPU scheduling in C.

```
# include <stdio.h>
int main()
{
  int n, pid[20], at[20], bt[20], ct[20], tat[20],
  wt[20];

  printf("Enter total numbers of processes: ");
  scanf("%d", &n);

  for (int i=0; i<n; i++)
  { at [i]=0;
     pid [i] = i+1;
     printf ("Burst time of P%d: ", i+1);
     scanf ("%d", &bt[i]);
  }
  for (int i= 0; i<n; i++)
  {  for (int j =i+1; j<n; j++)
     {  if (bt[i] > bt[j])
        { int temp = bt [i];
              bt [i] = bt[j];
              bt [j] = temp;

              temp = pid[i];
              pid [i] = pid [j];
              pid [j] = temp;
        }
     }
  }
```

Date: Page No.:

```
p[i] = p [j];
p [j] = temp;
}
}
}

int time = 0;
for (i=0; i<n; i++){
    if (time < p[i].at)
        time = p[i].at);

    p[i].wt = time - p [i].at;
    time += p[i].bt;
    p[i].tat = p [i].wt + p[i].bt;
}

float total_wt = 0, total_tat = 0;

printf("\PID \t AT \t BT \t WT \t TAT \n ");
for (i=0; i<n; i++){
    printf(P %d \t %d \t %d \t %d \t %d \n"
        p[i].pid, p[i].at, p[i].bt, p[i].wt, p[i].tat);
    total_wt += p [i].wt;
    total_tat += p [i].tat;
}
printf("Average TAT is %.2f \n", (float) sum 2/n);
printf("Average waiting time is %.2f \n", (float) sum 3/n);
return 0;
}}
```

Date:

Experiment :- 02.

Page No.:

AIM:-

To implement FCFS Cpu Scheduling in C.

```
#include <stdio.h>
struct process {
        int pid;
        int at;
        int bt;
        int wt;
        int tat;
    };

int main() {
    int n, i, j;
    struct process p[50], temp;
    printf("Enter number of process");
    Scanf("%d", &n);

    for (i=0; i<n; i++) {
        p[i].pid = i+1
        printf("\n process P%d Arival time: ", P[i].pid);
        Scanf("%d", &p[i].at);
        printf("process P%d Burst Time: ", P[i].Pid);
        Scanf("%d", &P[i].bt);
    }

for (i=0; i<n-1; i++){
    for (j=i+1; j<n; j++) {
        if (P[i].at > P[j].at){
            temp = P[i];
```

Date:

Page No.:

☆ Permission Commands:-

1. Chmod:- Change file permission.
2. Chown:- Change file owner.

☆ System Info Commands:-

1. date:- Show current date & time
2. Cal:- Show Calender
3. whoami:- Show current user
4. who:- Show logged in users.
5. Uname:- Show system info.

AIM:- To implement SJF CPU scheduling in C.

```
#include <stdio.h>
int main()
{
int n, pid[20], at[20], bt[20], ct[20], tat[20], wt[20];

printf("Enter total number of processes: ");
scanf("%d", &n);

for (int i=0; i<n; i++)
{ at[i]=0;
  pid[i]= i+1;
  printf("Burst time of P%d: ", i+1);
  scanf("%d", & bt[i]);
}

for(int i=0; i<n; i++)
{ for (int j= i+1; j<n; j++)
  { if (bt[i] × bt[j])
    { int temp = bt[i];
      bt[i] = bt[j];
      bt[j] = temp;

      temp = pid[i];
      pid[i] = pid[j];
      pid[j] = temp;
    }
  }
}
```

```
ct[0] = 0;

for (int i=0; i<n; i++)
{ ct[i] += bt[i];
  ct[i+1] = tat[i] = ct[i];
  wt[i] = tat[i] - bt[i];
}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0; i<n; i++)
{ printf("%d\t%d\t%d\t%d\t%d\t%d\n",
          pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
}

int sum=0 ; sum2 =0;
for(int i=0; i<n; i++)
{ sum += tat[i];
  sum2+= wt[i];
}

printf("Average Turn Around Time is %.2f.\n",
          (float) sum/n);

printf("Average Waiting Time is %.2f.\n",
          (float) sum2/n);
return0;
}
```

Implementation of Round Robin scheduling using C.

```
#include <stdio.h>
int main()
{
 int n, pid[20], at[20], bt[20], ct[20], tat[20];
 int wt[20], tq=2, remain, time, rt[20];

 printf("Enter number of processes : ");
 scanf("%d", &n);

 for (int i=0; i<n; i++)
 { pid[i] = i+1;
   at[i] = 0;
   printf("Burst time of P%d:", i+1);
   scanf("%d", &bt[i]);
   rt[i] = bt[i];
 }

  remain = n;
  time = 0;
  while (remain != 0)
  { for (int i=0; i<n; i++)
     { if (rt[i] > 0)
        { if (rt[i] <= tq)
           { time += rt[i];
             rt[i] = 0;
             ct[i] = time;
             remain--;
           }
```

```
            else
            { rt[i] -= tq;
              time t = tq;
            }
          }
        }
  }
  for (int i=0; i<n; i++)
  { tat[i] = ct[i];
     wt[i] = tat[i]
  }
  printf("Pid\tAT\tBT\tCT\tTAT\tWT\n");
  for (int i=0; i<n; i++)
  { printf("%d\t%d\t%d\t%d\t%d\t%d\t",
                pid[i], at[i], bt[i], ct[i], tat[i],
                 wt[i]);
   }
   int sum=0; sum2=0;
   for(int i=0; i<n; i++)
    { sum += tat[i];
       sum2 += wt[i];
    }

   printf("Average Turn around Time = %.2f\n",
                   (float) sum/n);
   printf("Average Waiting Time = %2f", (float)sum2/n);

   return 0;
}
```